タスク登録タブ UI仕様書

概要

目的

プロジェクトに登録されたMAIMLファイルの各メソッドに対して、実験条件・試行回数・作業順序を設定し、タスクとして登録するタブコンポーネント。タスクごとに指示を追加し、一括保存してワーカーに作業を割り当てる準備を行う。

使用場面

- プロジェクト管理画面の「タスク登録」タブ
- MAIML登録後、実験タスクの詳細設定を行う際に使用
- 実験管理者が実験条件・試行回数・作業順序を設定し、ワーカーに対する指示を登録する

アクセス権限

- **Experiment Admin（実験管理者）**: タスク登録・編集・保存
- **その他のロール**: アクセス不可

画面構成

レイアウト構造

```
┌──────────────────────────────────────────────────────┐
│ [ドラッグ&ドロップでタスクの順序を変更できます]              │
│                                    [プロジェクト指示] [保存] │
├──────────────────────────────────────────────────────┤
│ ┌──┬────┬──────┬──────────┬──────┬────┬──────┐ │
│ │☑│ ⁝⁝ │ 作業順序 │ メソッド │ 実験条件 │試行回数│ 指示 │ │
│ ├──┼────┼──────┼──────────┼──────┼────┼──────┤ │
│ │☑│ ⁝⁝ │  [1▼]  │ Method1  │[選択▼] │ [3]  │[+指示]│ │
```

```
| |☑| ⁝⁝ |  [2▼]  | Method2  |[選択▼] | [1]  |[+指示]| |
| |☑| ⁝⁝ |  [3▼]  | Method3  |[選択▼] | [5]  |[+指示]| |
| └──┴────┴──────┴──────────┴────────┴─────┴──────┘ |
└──────────────────────────────────────────────────┘

【モーダル】
┌──────────────────────────────────┐
│ 指示追加                    [×]│
├──────────────────────────────────┤
│ 対象: [メソッド選択▼]             │
│ 種別: [選択▼]                    │
│ 優先度: [選択▼]                  │
│ 内容: [テキストエリア]            │
│                                  │
│          [キャンセル] [保存]      │
└──────────────────────────────────┘
```

UI要素

1. ツールバー

- **ヒントテキスト**:「ドラッグ&ドロップでタスクの順序を変更できます」(左揃え)
- **プロジェクト指示ボタン**: プロジェクト全体に対する指示を登録
- **保存ボタン**: チェックされたタスクと指示を一括保存

2. タスク一覧テーブル

カラム	説明	幅	編集可否
チェックボックス	タスク選択（保存対象）	50px	可
ドラッグハンドル	⁝⁝（ドラッグ可能アイコン）	50px	可
作業順序	1〜N（Select形式）	120px	可
メソッド	メソッドID・名称（読み取り専用）	-	不可
実験条件	0（テスト条件）/ 1（実験条件）（Select形式）	200px	可
試行回数	1〜999（数値入力）	120px	可
指示	「+指示」ボタン（指示追加ダイアログを開く）	120px	可

3. 非活性条件

以下の場合、行全体が非活性（グレーアウト）：

- プロジェクトがチェックアウトされている（`isCheckoutLocked = true`）
- 保存処理中（`isSaving = true`）
- データ読み込み中（`loadingMethods = true`）
- **ワーカー連携済み**（`workerLinkedFlg = true`）← 既にワーカーに送信済みのタスク

4. ドラッグ&ドロップ機能

- 各行をドラッグして順序を入れ替え可能
- **制約**: ワーカー連携済みタスク（`workerLinkedFlg = true`）は移動不可
- ドラッグ対象またはドロップ先がワーカー連携済みの場合は警告メッセージを表示

機能仕様

Props定義

```
interface TaskRegistrationTabProps {
  projectId: string;                    // プロジェクトID
  projectName: string;                  // プロジェクト名
  isActive: boolean;                    // タブがアクティブか
  isCheckoutLocked?: boolean;           // チェックアウト状態（親から渡される）
  onTasksChange?: (tasks: TaskDetail[]) => void; // タスク変更時のコールバック
}
```

State管理

主要なState

```
// メソッド情報
const [availableMethods, setAvailableMethods] = useState<MethodInfo[]>([]);
const [loadingMethods, setLoadingMethods] = useState<boolean>(false);
```

```
// タスク詳細
const [taskDetails, setTaskDetails] = useState<TaskDetail[]>([]);

// 保存状態
const [isSaving, setIsSaving] = useState<boolean>(false);
const [isMaimlRegistered, setIsMaimlRegistered] = useState<boolean>(false);

// チェックボックス状態（タスクIDで管理）
const [checkedTaskIds, setCheckedTaskIds] = useState<Set<number>>(new Set());

// 指示管理
const [instructions, setInstructions] = useState<InstructionData[]>([]);
const [isInstructionDialogOpen, setIsInstructionDialogOpen] = useState<boolean>(false);
const [selectedMethodIdForInstruction, setSelectedMethodIdForInstruction] = useState<string>
('');
const [availableTransitions, setAvailableTransitions] = useState<TransitionInfo[]>([]);

// プロジェクト指示用のダイアログ状態
const [isProjectInstructionDialogOpen, setIsProjectInstructionDialogOpen] = useState<boolean>
(false);
```

チェックアウト状態

- Propsで渡された `isCheckoutLocked` を優先
- なければfalse（デフォルト）

Context連携

```
const { t } = useLanguage();                    // 多言語対応
const { maimlRoot } = useProjectMaiml();        // MAIMLデータ取得
const { options: experimentConditionOptions, loading: loadingCodeMaster } =
useCodeMaster('experiment_condition'); // コードマスター
```

主要機能

1. タスクデータの取得（初回表示・タブアクティブ時）

実行タイミング: タブがアクティブになった時（ `isActive = true` ）

処理フロー:

```
useEffect(() => {
  if (!projectName || !isActive) return;

  // 1. タスク登録データをAPI取得
  const taskData = await getTaskRegistration(projectName);

  // 2. MAIML登録状態を保存
  setIsMaimlRegistered(taskData.isMaimlRegistered);

  // 3. 指示データをリフレッシュ（stateクリア + DB再取得）
  await refreshInstructions();

  // 4. タスク情報をUI用に変換
  if (taskData.tasks && taskData.tasks.length > 0) {
    // DBにデータがある場合: 既存データを表示
    const uiTasks = taskData.tasks.map((task, index) => ({
      id: index + 1,
      methodId: task.methodId,
      experimentCondition: Number(task.experimentCondition),
      trialCount: task.trialCount,
      experimentOrder: task.experimentOrder,
      taskNumberOptions: Array.from({ length: task.trialCount }, (_, i) => i + 1),
      workerLinkedFlg: task.workerLinkedFlg || false,
    }));

    setTaskDetails(uiTasks);

    // worker_linked_flg=falseのタスクIDでcheckedTaskIdsを初期化
    const editableTaskIds = new Set(uiTasks.filter(t => !t.workerLinkedFlg).map(t => t.id));
    setCheckedTaskIds(editableTaskIds);
  } else if (maimlRoot?.protocol?.methods) {
    // DBにデータがない場合: MAIMLコンテキストから初期値を生成
    const methods = maimlRoot.protocol.methods;
    const initialTasks = methods.map((method, index) => ({
      id: index + 1,
      methodId: method.id,
      experimentCondition: 0,
      trialCount: 1,
      experimentOrder: index + 1,
      taskNumberOptions: [1],
      workerLinkedFlg: false,
    }));

    setTaskDetails(initialTasks);

    // 全タスクIDでcheckedTaskIdsを初期化（初回）
    const allTaskIds = new Set(initialTasks.map(t => t.id));
    setCheckedTaskIds(allTaskIds);
```

```
  }
}, [projectName, isActive, refreshInstructions]);
```

API: `GET /api/v1/experiment-admin/project/:projectName/task-registration`

レスポンス:

```
{
  isMaimlRegistered: boolean;
  tasks: Array<{
    methodId: string;
    experimentCondition: string;
    trialCount: number;
    experimentOrder: number;
    workerLinkedFlg: boolean;
  }>;
}
```

2. チェックボックスの状態管理

初期化:

- DBにデータがある場合: `workerLinkedFlg = false` のタスクIDのみチェック
- DBにデータがない場合: 全タスクIDをチェック

変更ハンドラー:

```
const handleCheckboxChange = useCallback((taskId: number, checked: boolean) => {
  setCheckedTaskIds(prev => {
    const newSet = new Set(prev);
    if (checked) {
      newSet.add(taskId);
    } else {
      newSet.delete(taskId);
    }
    return newSet;
  });
}, []);
```

3. タスク詳細の更新

```
const handleUpdateTaskDetail = useCallback(
  (id: number, field: keyof TaskDetail, value: string | number) => {
    setTaskDetails((prev) =>
      prev.map((task) => {
        if (task.id === id) {
          if (field === 'experimentCondition') {
            return { ...task, [field]: Number(value) as 0 | 1 };
          }

          if (field === 'trialCount') {
            const newTrialCount = typeof value === 'number' ? value :
parseInt(value.toString(), 10);
            if (!isNaN(newTrialCount) && newTrialCount > 0) {
              const newOptions = Array.from({ length: newTrialCount }, (_, i) => i + 1);
              return { ...task, trialCount: newTrialCount, taskNumberOptions: newOptions };
            }
          }

          return { ...task, [field]: value };
        }
        return task;
      })
    );
  },
  []
);
```

4. ドラッグ&ドロップによる順序変更

ライブラリ: `@dnd-kit/core`, `@dnd-kit/sortable`

```
const handleDragEnd = useCallback((event: DragEndEvent) => {
  const { active, over } = event;

  if (over && active.id !== over.id) {
    setTaskDetails((items) => {
      const oldIndex = items.findIndex((item) => item.id === active.id);
      const newIndex = items.findIndex((item) => item.id === over.id);

      const draggedItem = items[oldIndex];
      const targetItem = items[newIndex];
```

```
      // ワーカー連携済みタスクは移動不可
      if (draggedItem?.workerLinkedFlg || targetItem?.workerLinkedFlg) {
        message.warning(t('taskRegistration.messages.cannotChangeOrder'));
        return items;
      }

      return arrayMove(items, oldIndex, newIndex);
    });
  }
}, []);
```

制約:

- ドラッグ対象または移動先が `workerLinkedFlg = true` の場合は順序変更不可
- 警告メッセージ: `taskRegistration.messages.cannotChangeOrder`

5. 作業順序の手入力変更

```
const handleOrderChange = useCallback((taskId: number, newOrder: number) => {
  setTaskDetails((prev) => {
    const currentIndex = prev.findIndex((task) => task.id === taskId);
    if (currentIndex === -1) return prev;

    const newIndex = newOrder - 1;

    // 範囲外チェック
    if (newIndex < 0 || newIndex >= prev.length) {
      message.warning(t('taskRegistration.messages.invalidOrder').replace('{max}',
prev.length.toString()));
      return prev;
    }

    // 同じ位置なら変更不要
    if (currentIndex === newIndex) return prev;

    const movingItem = prev[currentIndex];
    const targetItem = prev[newIndex];

    // ワーカー連携済みタスクは移動不可
    if (movingItem?.workerLinkedFlg || targetItem?.workerLinkedFlg) {
      message.warning(t('taskRegistration.messages.cannotChangeOrder'));
      return prev;
    }

    return arrayMove(prev, currentIndex, newIndex);
```

```
  });
}, [t]);
```

作業順序のSelectオプション:

- `workerLinkedFlg = true` のタスク: 現在位置のみ選択可能
- `workerLinkedFlg = false` のタスク: ワーカー連携済みタスク以外の位置に移動可能

6. 指示追加ボタン

タスク指示追加:

```
const handleAddInstructionClick = useCallback((methodId: string) => {
  setSelectedMethodIdForInstruction(methodId);
  setIsInstructionDialogOpen(true);
}, []);
```

プロジェクト指示追加:

```
const handleProjectInstructionClick = useCallback(() => {
  setIsProjectInstructionDialogOpen(true);
}, []);
```

指示保存ハンドラー:

```
const handleInstructionSave = useCallback((instruction: InstructionData, keepDialogOpen?:
boolean) => {
  // instruction_status=0（未送信）として追加
  const newInstruction: InstructionData = {
    ...instruction,
    instruction_status: 0,
    isNewlyAdded: true // 新規追加フラグを設定
  };

  setInstructions(prev => [...prev, newInstruction]);
  message.success(t('instructionRegistration.messages.added'));

  if (!keepDialogOpen) {
    setIsInstructionDialogOpen(false);
```

```
    setIsProjectInstructionDialogOpen(false);
    setSelectedMethodIdForInstruction('');
  }
}, []);
```

ダイアログ種別:

- タスク指示ダイアログ: `fixedMethodId={selectedMethodIdForInstruction}`（特定メソッドに紐付く）
- プロジェクト指示ダイアログ: `fixedMethodId=""`（プロジェクト全体に対する指示）

7. 保存処理

実行フロー:

```
const handleSave = useCallback(async () => {
  try {
    // 1. バリデーション: 未送信指示のチェック
    const unsentInstructions = instructions.filter(inst => inst.instruction_status === 0);
    if (unsentInstructions.length === 0) {
      message.warning(t('taskRegistration.messages.requireInstruction'));
      return;
    }

    // 2. チェックされたタスクのチェック
    if (checkedTaskIds.size === 0) {
      message.warning(t('taskRegistration.messages.selectMethods'));
      return;
    }

    // 3. 初回保存かどうかを判定
    const currentData = await getTaskRegistration(projectName);
    const isFirstSave = !currentData.tasks || currentData.tasks.length === 0;

    // 4. 初回保存時は全タスクがチェックされているか確認
    if (isFirstSave) {
      const allTaskIds = taskDetails.map(t => t.id);
      const allChecked = allTaskIds.every(id => checkedTaskIds.has(id));

      if (!allChecked) {
        message.warning(t('taskRegistration.messages.selectAllMethodsFirst'));
        return;
      }
    }
```

```
// 5. チェックされた かつ workerLinkedFlg=false のタスクをフィルタリング
const checkedTasks = taskDetails.filter(task =>
  checkedTaskIds.has(task.id) && !task.workerLinkedFlg
);

// 6. バリデーション: メソッドID、試行回数
const emptyMethod = checkedTasks.find((task) => !task.methodId);
if (emptyMethod) {
  message.warning(t('taskRegistration.messages.selectAllMethods'));
  return;
}

const invalidTrialCount = checkedTasks.find(
  (task) => task.trialCount < 1 || task.trialCount > 999
);
if (invalidTrialCount) {
  message.warning(t('taskRegistration.messages.invalidTrialCount'));
  return;
}

// 7. チェックアウト状態チェック
if (isCheckoutLocked) {
  message.warning(t('taskRegistration.messages.checkedOut'));
  return;
}

// 8. MAIML登録状態チェック
if (!isMaimlRegistered) {
  message.warning(t('taskRegistration.messages.maimlNotRegistered'));
  return;
}

setIsSaving(true);

// 9. 新規追加した指示のみを送信対象としてフィルタリング
const checkedMethodIds = new Set(checkedTasks.map(t => t.methodId));
const checkedInstructions = instructions.filter((inst: InstructionData) =>
  inst.isNewlyAdded === true &&
  (inst.methodId === '' || checkedMethodIds.has(inst.methodId))
);

// 10. API用の指示データを作成
const instructionSaveItems = checkedInstructions.map((inst: InstructionData) => ({
  methodId: inst.methodId,
  content: inst.content,
  priority: inst.priority,
  type: inst.type,
  targetType: inst.targetType,
  transitionId: inst.transitionId || null,
  taskNumber: inst.taskNumber || null,
```

```
    }));

    // 11. workerLinkedFlg=trueの既存タスクの最大experiment_orderを取得
    const maxExistingOrder = taskDetails
      .filter(t => t.workerLinkedFlg)
      .reduce((max, t) => Math.max(max, t.experimentOrder || 0), 0);

    // 12. API用のリクエストデータを作成
    const requestData = {
      tasks: checkedTasks.map((task, index) => ({
        methodId: task.methodId,
        experimentCondition: task.experimentCondition,
        trialCount: task.trialCount,
        experimentOrder: maxExistingOrder + index + 1, // 既存の最大order + 連番
      })),
      instructions: instructionSaveItems.length > 0 ? instructionSaveItems : undefined,
    };

    // 13. APIを呼び出し
    const response = await saveTaskRegistration(projectName, requestData);

    // 14. 保存成功後、指示データをリフレッシュ（stateクリア + DB再取得）
    await refreshInstructions();

    onTasksChange?.(checkedTasks);
    message.success(response.message || t('taskRegistration.messages.saved'));
  } catch (error) {
    const errorMessage =
      error instanceof Error ? error.message : t('taskRegistration.messages.saveFailed');
    message.error(errorMessage);
    console.error('Failed to save task registration:', error);
  } finally {
    setIsSaving(false);
  }
}, [taskDetails, checkedTaskIds, instructions, projectName, isCheckoutLocked,
isMaimlRegistered, refreshInstructions, onTasksChange, t]);
```

API: `POST /api/v1/experiment-admin/project/:projectName/task-registration`

リクエスト:

```
{
  tasks: Array<{
    methodId: string;
    experimentCondition: number; // 0 or 1
    trialCount: number;          // 1〜999
```

```
    experimentOrder: number;     // 作業順序
  }>;
  instructions?: Array<{
    methodId: string;
    content: string;
    priority: string;
    type: string;
    targetType: string;
    transitionId: string | null;
    taskNumber: number | null;
  }>;
}
```

バリデーション:

- 未送信指示が1件以上あること
- チェックされたタスクが1件以上あること
- 初回保存時は全タスクがチェックされていること
- 全タスクにメソッドIDが設定されていること
- 試行回数が1〜999の範囲内であること
- チェックアウトされていないこと
- MAIML登録済みであること

保存後の処理:

- 指示データをリフレッシュ（`refreshInstructions()`）
- `onTasksChange` コールバック実行
- 成功メッセージ表示

8. 指示データのリフレッシュ

```
const refreshInstructions = useCallback(async () => {
  setInstructions([]); // stateをクリア
  const instructionsData = await getInstructions(projectName).catch(err => {
    console.warn('[DEBUG] Failed to refresh instructions (continuing):', err);
    return [];
  });
  setInstructions(instructionsData || []);
}, [projectName]);
```

実行タイミング:

- タブアクティブ時（初回表示）
- 保存処理成功後

API: `GET /api/v1/experiment-admin/project/:projectName/instructions`

データ型定義

TaskDetail

```
interface TaskDetail {
  id: number;                          // UI内部ID
  methodId: string;                    // メソッドID
  experimentCondition: 0 | 1;          // 実験条件（0: テスト条件, 1: 実験条件）
  trialCount: number;                  // 試行回数（1〜999）
  experimentOrder: number;             // 作業順序
  taskNumberOptions: number[];         // タスク番号のオプション（1〜trialCount）
  workerLinkedFlg: boolean;            // ワーカー連携済みフラグ
}
```

MethodInfo

```
interface MethodInfo {
  id: string;                          // メソッドID
  name: string;                        // メソッド名
  displayName: string;                 // 表示名
  description?: string;                // 説明
}
```

InstructionData

```
interface InstructionData {
  methodId: string;                    // メソッドID（空文字の場合はプロジェクト指示）
```

```
  content: string;                    // 指示内容
  priority: string;                   // 優先度
  type: string;                       // 種別
  targetType: string;                 // 対象種別
  transitionId?: string | null;       // TransitionID
  taskNumber?: number | null;         // タスク番号
  instruction_status: number;         // 指示ステータス（0: 未送信, 1: 送信済み）
  isNewlyAdded?: boolean;             // 新規追加フラグ
}
```

TaskRegistrationData（APIレスポンス）

```
interface TaskRegistrationData {
  isMaimlRegistered: boolean;         // MAIML登録状態
  tasks: Array<{
    methodId: string;
    experimentCondition: string;      // "0" or "1"（文字列）
    trialCount: number;
    experimentOrder: number;
    workerLinkedFlg: boolean;
  }>;
}
```

InstructionSaveItem（API送信用）

```
interface InstructionSaveItem {
  methodId: string;
  content: string;
  priority: string;
  type: string;
  targetType: string;
  transitionId: string | null;
  taskNumber: number | null;
}
```

子コンポーネント

InstructionEditDialog

用途: タスク指示・プロジェクト指示の追加ダイアログ

Props:

```
{
  open: boolean;                        // ダイアログ表示状態
  mode: 'create' | 'edit';              // モード（作成/編集）
  tasks: TaskDetail[];                  // タスク一覧
  availableTransitions: TransitionInfo[]; // Transition一覧
  onClose: () => void;                  // 閉じるハンドラー
  onSave: (instruction: InstructionData, keepDialogOpen?: boolean) => void; // 保存ハンドラー
  fixedMethodId: string;                // 固定メソッドID（""の場合はプロジェクト指示）
  saveMode: 'local';                    // 保存モード（'local': stateに保存のみ）
}
```

使用箇所:

- タスク指示追加: `fixedMethodId={selectedMethodIdForInstruction}`
- プロジェクト指示追加: `fixedMethodId=""`

DraggableRow

用途: ドラッグ可能な行コンポーネント

Props:

```
interface DraggableRowProps extends React.HTMLAttributes<HTMLTableRowElement> {
  'data-row-key': string;
}
```

実装:

```
const DraggableRow = (props: DraggableRowProps) => {
  const { children, ...restProps } = props;
  const {
    attributes,
    listeners,
```

```
    setNodeRef,
    transform,
    transition,
    isDragging,
  } = useSortable({
    id: restProps['data-row-key'],
  });

  const rowKey = restProps['data-row-key'];
  const rowRecord = taskDetails.find(task => task.id.toString() === rowKey.toString());
  const isRowWorkerLinked = rowRecord?.workerLinkedFlg || false;

  const style = {
    ...restProps.style,
    transform: CSS.Transform.toString(transform),
    transition,
    ...(isDragging ? { position: 'relative', zIndex: 9999 } : {}),
    ...(isRowWorkerLinked ? {
      backgroundColor: '#f5f5f5',
      opacity: 0.7
    } : {}),
  };

  return (
    <tr {...restProps} ref={setNodeRef} style={style} {...attributes}>
      {React.Children.map(children, (child) => {
        if ((child as React.ReactElement).key === 'sort') {
          const isDragDisabled = isCheckoutLocked || isRowWorkerLinked;
          return React.cloneElement(child as React.ReactElement, {}, (
            <div
              {...(isDragDisabled ? {} : listeners)}
              style={{
                cursor: isDragDisabled ? 'not-allowed' : 'grab',
                padding: '8px',
                opacity: isDragDisabled ? 0.5 : 1
              }}
            >
              <HolderOutlined />
            </div>
          ));
        }
        return child;
      })}
    </tr>
  );
};
```

エラーハンドリング

エラーメッセージ

エラー条件	メッセージキー	表示メッセージ
未送信指示が0件	`taskRegistration.messages.requireInstruction`	「指示を1件以上登録してください」
チェックされたタスクが0件	`taskRegistration.messages.selectMethods`	「保存するメソッドを選択してください」
初回保存時に全タスクがチェックされていない	`taskRegistration.messages.selectAllMethodsFirst`	「初回保存時は全メソッドを選択してください」
メソッドIDが未設定	`taskRegistration.messages.selectAllMethods`	「全タスクにメソッドを選択してください」
試行回数が範囲外	`taskRegistration.messages.invalidTrialCount`	「試行回数は1〜999の範囲で設定してください」
チェックアウト中	`taskRegistration.messages.checkedOut`	「チェックアウト中は保存できません」
MAIML未登録	`taskRegistration.messages.maimlNotRegistered`	「MAIMLを登録してください」
ワーカー連携済みタスクの順序変更	`taskRegistration.messages.cannotChangeOrder`	「ワーカー連携済みタスクの順序は変更できません」
作業順序が範囲外	`taskRegistration.messages.invalidOrder`	「作業順序は1〜{max}の範囲で設定してください」
保存失敗	`taskRegistration.messages.saveFailed`	「保存に失敗しました」

ローディング状態

メソッド読み込み中:

```
<Spin size="large" tip={t('taskRegistration.loading')}>
  <div style={{ minHeight: 100 }} />
</Spin>
```

メソッドが0件:

```
<Text type="secondary">{t('taskRegistration.noMethods')}</Text>
```

保存中:

- 保存ボタンに `loading` プロップを設定
- ボタンテキスト: `taskRegistration.saving`

特記事項

ワーカー連携済みタスクの扱い

workerLinkedFlg = true のタスク:

- チェックボックス: 非活性（グレーアウト）
- ドラッグハンドル: 非活性（`not-allowed` カーソル）
- 実験条件・試行回数: 非活性
- 指示追加ボタン: 非活性
- 作業順序: 現在位置のみ選択可能（移動不可）
- 行背景色: `#f5f5f5`（グレー）
- 透明度: `0.7`

保存時の扱い:

- 保存対象外（APIには送信しない）
- 既存の `experiment_order` を維持
- 新規タスクの `experiment_order` は、ワーカー連携済みタスクの最大値 + 連番

初回保存時の制約

初回保存時（DBにデータがない場合）:

- 全メソッドをチェックすることが必須
- 一部のメソッドのみをチェックして保存することは不可
- メッセージ: `taskRegistration.messages.selectAllMethodsFirst`

2回目以降の保存時:

- 任意のメソッドをチェックして保存可能
- 新規追加分のみがDBに保存される

指示の保存方式

ローカルstate管理:

- 指示追加ボタンクリック → InstructionEditDialog表示
- ダイアログで指示を入力 → `onSave` でstateに追加（`isNewlyAdded: true`）
- 保存ボタンクリック → 新規追加フラグ付き指示のみをAPIに送信

DB保存後のリフレッシュ:

- 保存成功後、`refreshInstructions()` を実行
- stateをクリア → DBから再取得
- これにより、`instruction_status = 0`（未送信）の指示と`instruction_status = 1`（送信済み）の指示を区別

実験条件の型変換

APIレスポンス: `experimentCondition` は文字列（`"0"` or `"1"`） **UI内部**: `experimentCondition` は数値型（`0` or `1`）

```
// API → UI変換
experimentCondition: Number(task.experimentCondition) as 0 | 1

// UI → API変換
experimentCondition: task.experimentCondition // 数値のまま送信
```

作業順序の計算

新規タスクの `experimentOrder`:

```
const maxExistingOrder = taskDetails
  .filter(t => t.workerLinkedFlg)
  .reduce((max, t) => Math.max(max, t.experimentOrder || 0), 0);

const newOrder = maxExistingOrder + index + 1;
```

例:

- 既存タスク（ワーカー連携済み）: `experimentOrder = 1, 2, 3`
- 新規タスク3件を保存 → `experimentOrder = 4, 5, 6`

API連携

1. タスク登録データ取得

エンドポイント: `GET /api/v1/experiment-admin/project/:projectName/task-registration`

レスポンス:

```
{
  isMaimlRegistered: boolean;
  tasks: Array<{
    methodId: string;
    experimentCondition: string;       // "0" or "1"
    trialCount: number;
    experimentOrder: number;
    workerLinkedFlg: boolean;
  }>;
}
```

2. タスク登録保存

エンドポイント: `POST /api/v1/experiment-admin/project/:projectName/task-registration`

リクエスト:

```
{
  tasks: Array<{
    methodId: string;
    experimentCondition: number;       // 0 or 1
    trialCount: number;
    experimentOrder: number;
  }>;
  instructions?: Array<{
    methodId: string;
    content: string;
    priority: string;
    type: string;
    targetType: string;
    transitionId: string | null;
    taskNumber: number | null;
  }>;
}
```

レスポンス:

```
{
  message: string;
}
```

3. 指示データ取得

エンドポイント: `GET /api/v1/experiment-admin/project/:projectName/instructions`

レスポンス:

```
Array<{
  methodId: string;
  content: string;
  priority: string;
  type: string;
  targetType: string;
  transitionId?: string | null;
  taskNumber?: number | null;
  instruction_status: number;          // 0: 未送信, 1: 送信済み
}>
```

実装ファイル情報

パス

```
frontend/src/components/project/TaskRegistrationTab.tsx
```

主要な依存パッケージ

- `antd` : UI コンポーネント（Table, Button, Select, Input, Space, Typography, theme, Spin, Empty, App）
- `@ant-design/icons` : アイコン（SaveOutlined, HolderOutlined, PlusOutlined）
- `@dnd-kit/core` : ドラッグ&ドロップ機能（DndContext, sensors）
- `@dnd-kit/sortable` : ソート可能リスト（SortableContext, useSortable, arrayMove）
- `react` : React本体（useState, useEffect, useCallback, useMemo）

Context

- `LanguageContext` : 多言語対応
- `ProjectMaimlContext` : MAIMLデータ取得
- `useCodeMaster` : コードマスターデータ取得

API Service

- `projectApiService` : タスク登録・指示関連のAPI呼び出し

Utils

- `selectUtils` : Selectオプション生成関数

変更履歴

バージョン	日付	変更内容	担当者
1.0	2026-03-11	実装ベースで仕様書を新規作成（実装ファイル: TaskRegistrationTab.tsx）	Cline

---

関連ドキュメント

- プロジェクト管理画面
- 指示登録タブ
- 指示編集ダイアログ
- タスク結果一覧テーブル